

assignment-04-davis-club

March 14, 2026

0.0.1 Assignment Four - Davis Club

Executive Summary

This project analyzes the Davis Southern Women dataset, which records the attendance of 18 women at 14 social events connected by 89 participation ties, forming a bipartite social network. The instructor initially provided a single Python script and biadjacency matrix to construct and analyze the network. To improve clarity and understanding, the code was reorganized into modular sections that explain what each step does, why it matters, and how to interpret the results.

The analysis was expanded to include clearer visualizations and additional network metrics. I enhanced the graphs to highlight connectivity using color-coded nodes for women and events. Degree and weighted projections reveal a dense social core in which several women attended events alongside nearly all other participants. Event projections show that events E6–E9 function as central hubs, overlapping with almost all other gatherings. Betweenness centrality further identifies key social bridges, such as Evelyn Jefferson and Theresa Anderson, who connect different subgroups. Overall, the results reveal a core–periphery social structure linking central actors, events, and peripheral participants.

Instructions provided in Brightspace:

Here is a dataset that shows a simple 2-node network: the attendance of 18 Southern Women at 14 social events:

Brief Description. Small “musty” datasets like that from this 1941 study have proven very valuable in testing and comparing new network algorithms. Dataset. Python code to create dataset: https://networkx.github.io/documentation/stable/auto_examples/algorithms/plot_davis_club.html What can you infer about the relationships between (1) the women, and (2) the social events?

Please provide your code and analysis in an Jupyter notebook stored in your GitHub repository.

Davis Southern Club Women

The following code shows how to make unipartite projections of the graph and compute the properties of those graphs.

These data were collected by Davis et al. in the 1930s. They represent observed attendance at 14 social events by 18 Southern women. The graph is bipartite (clubs, women).

```
[1]: # import matplotlib.pyplot as plt
      # import networkx as nx
      # from networkx.algorithms import bipartite
```

```

# G = nx.davis_southern_women_graph()
# women = G.graph["top"]
# clubs = G.graph["bottom"]

# print("Biadjacency matrix")
# print(bipartite.biadjacency_matrix(G, women, clubs))

# # project bipartite graph onto women nodes
# W = bipartite.projected_graph(G, women)
# print()
# print("#Friends, Member")
# for w in women:
#     print(f"{W.degree(w)} {w}")

# # project bipartite graph onto women nodes keeping number of co-occurrence
# # the degree computed is weighted and counts the total number of shared
# ↪ contacts
# W = bipartite.weighted_projected_graph(G, women)
# print()
# print("#Friend meetings, Member")
# for w in women:
#     print(f"{W.degree(w, weight='weight')} {w}")

# pos = nx.spring_layout(G, seed=648) # Seed layout for reproducible node
# ↪ positions
# nx.draw(G, pos)
# plt.show()

```

0.0.2 Assignment Structure

I created a simple Modular structure to help me understand the issues and the Python code to address them. Each module addresses what the Python code will do, why is it important and how to interpret the output.

I took the instructor-provided Python code and separated it into modules 1 through 5. I then added my additional analysis in modules 5A, 6, 6A, 7 and 7A.

=====

0.0.3 Model 1: Preparing the Tools

In this section, I loaded the software libraries needed to complete my analysis.

In sum, these tools allow me to:

- Build and analyze a network.
- Create visual charts to help interpret results.

Even though some of these libraries are basic and repeatedly used in many projects, such as pandas, I commented out each library to understand their functions.

```
[2]: # Module 1: Import Libraries

import pandas as pd                # Used to work with tables of data (like
    ↪ Excel sheets in Python)

import numpy as np                 # Used for numerical calculations and
    ↪ handling arrays of numbers

import networkx as nx              # Used to build and analyze a network
    ↪ structure

from networkx.algorithms import bipartite # These functions are designed to
    ↪ analyze two-mode networks between groups.

from matplotlib.patches import Patch # Used to create a graphical object used
    ↪ to draw shapes and legends.

import matplotlib.pyplot as plt    # Used to control chart formatting and
    ↪ display plots

import math                        # Used to provide mathematical functions
    ↪ and constants used for numerical calculations.
```

0.0.4 Module 1A: Loading the Davis Southern Women Bipartite Network and Construction

What this code will do Chapter 5 describes these structures as 2-mode networks, where relationships exist between two different sets of nodes rather than within the same set. The first part of the instructor provided code block loads the Davis Southern Women dataset from NetworkX and extracts the two node sets:

- Women
- Social events

Why this code is important The dataset is a bipartite (two-mode) network, meaning:

- Women only connect to events
- Events only connect to women
- There are no direct woman-to-woman or event-to-event edges at this stage

Understanding the size and structure of the dataset provides context before performing projections or centrality analysis.

How to interpret the output

- 18 women = number of actors
- 14 events = number of social gatherings
- 89 edges = total attendance relationships

```
[3]: # Module 1A: Constructing the Bipartite Network

import networkx as nx
from networkx.algorithms import bipartite
import matplotlib.pyplot as plt

G = nx.davis_southern_women_graph()

women = G.graph["top"]
events = G.graph["bottom"]

print("Number of women:", len(women))
print("Number of events:", len(events))
print("Total edges (attendance ties):", G.number_of_edges())
```

```
Number of women: 18
Number of events: 14
Total edges (attendance ties): 89
```

0.0.5 Module 2: Constructing the Biadjacency Matrix

What this code will do The second part of the instructor provided code block converted the bipartite graph into a matrix form.

- Rows represent women.
- Columns represent events.
- A value of 1 indicates that a woman attended that event, a 0 means she did not.

Why this code is important The biadjacency matrix is the mathematical representation of the bipartite graph, allowing one to:

- See attendance patterns clearly
- Perform matrix-based operations
- Project the network into one-mode graphs

Chapter 5 explains that 2-mode networks can be represented mathematically as affiliation matrices, which capture membership relationships between individuals and groups. The biadjacency matrix converts the network into a table format.

How to interpret the output

- Each non-zero value represents a participation tie.
- The total number of stored elements (89) matches the number of edges in the graph.
- This matrix encodes the entire social participation structure.

```
[4]: # Module 2: Create Biadjacency Matrix

B = bipartite.biadjacency_matrix(G, women, events)
print(B)
```

```
<Compressed Sparse Row sparse array of dtype 'int64'
  with 89 stored elements and shape (18, 14)>
```

Coords	Values
(0, 0)	1
(0, 1)	1
(0, 2)	1
(0, 3)	1
(0, 4)	1
(0, 5)	1
(0, 7)	1
(0, 8)	1
(1, 0)	1
(1, 1)	1
(1, 2)	1
(1, 4)	1
(1, 5)	1
(1, 6)	1
(1, 7)	1
(2, 1)	1
(2, 2)	1
(2, 3)	1
(2, 4)	1
(2, 5)	1
(2, 6)	1
(2, 7)	1
(2, 8)	1
(3, 0)	1
(3, 2)	1
:	:
(12, 7)	1
(12, 8)	1
(12, 9)	1
(12, 11)	1
(12, 12)	1
(12, 13)	1
(13, 5)	1
(13, 6)	1
(13, 8)	1
(13, 9)	1
(13, 10)	1
(13, 11)	1
(13, 12)	1
(13, 13)	1
(14, 6)	1
(14, 7)	1
(14, 9)	1
(14, 10)	1
(14, 11)	1

(15, 7)	1
(15, 8)	1
(16, 8)	1
(16, 10)	1
(17, 8)	1
(17, 10)	1

[]:

0.0.6 Module 3: Project Bipartite Graph onto Women Nodes

What This Code Will Do This block projects the bipartite graph onto the women nodes.

Two women are connected if they attended at least one common event.

Why This Matters Projection allows us to study relationships between women directly.

Instead of analyzing attendance, I analyzed shared participation as a proxy for social interaction. Chapter 5 explains that researchers often project a 2-mode network into a 1-mode network in order to analyze relationships within one group.

How to Interpret the Output A higher degree means more social overlap. The output shows the degree of each woman in the projected network. It tells us how many other women each woman is socially connected to through shared event attendance.

Women with degree 17 are connected to nearly everyone. For example, Evelyn Jefferson attended events with 17 other women in the dataset (all but one person). Women with lower degrees (e.g., 11 or 12) are more peripheral.

This suggests:

- A highly cohesive core group
- Some variation in social participation
- Possible core-periphery structure

[5]: *# Module 3: Projecting the Network onto Women (Unweighted)*

```
W = bipartite.projected_graph(G, women)

print("#Friends, Member")
for w in women:
    print(W.degree(w), w)
```

```
#Friends, Member
17 Evelyn Jefferson
15 Laura Mandeville
17 Theresa Anderson
15 Brenda Rogers
11 Charlotte McDowd
15 Frances Anderson
15 Eleanor Nye
```

```
16 Pearl Oglethorpe
17 Ruth DeSand
17 Verne Sanderson
16 Myra Liddel
16 Katherina Rogers
17 Sylvia Avondale
17 Nora Fayette
17 Helen Lloyd
16 Dorothy Murchison
12 Olivia Carleton
12 Flora Price
```

0.0.7 Module 3A: Group Women With the Same Connection Level

What This Code Will Do This block groups women with the same number of connections.

How to Interpret the Output Seven women have 17 connections to other women, and 4 women have connections to 16 and 15 other women, showing this is a dense network.

```
[6]: # Module 3A: Group Women With the Same Connection Level
```

```
import pandas as pd
from networkx.algorithms import bipartite

# Prevent name truncation
pd.set_option('display.max_colwidth', None)

# Project the network onto women
W = bipartite.projected_graph(G, women)

# Build list of women and their degree
rows = []
for w in women:
    rows.append({
        "Woman": w,
        "Shared Event Connections": W.degree(w)
    })

df = pd.DataFrame(rows)

# Group women with the same connection count
grouped = (
    df.groupby("Shared Event Connections")["Woman"]
      .apply(lambda x: ", ".join(sorted(x)))
      .reset_index()
)

# Sort descending
```

```

grouped = grouped.sort_values(
    by="Shared Event Connections",
    ascending=False
).reset_index(drop=True)

# Rename column
grouped = grouped.rename(columns={"Woman": "Women"})

# Style: left align headers and women names
grouped.style \
    .set_properties(subset=["Women"], **{'text-align': 'left'}) \
    .set_table_styles([
        {'selector': 'th',
         'props': [('text-align', 'left')]}
    ])

```

[6]: <pandas.io.formats.style.Styler at 0x19fca4d3380>

0.0.8 Module 4: Project Bipartite Graph onto Women Nodes Keeping Number of Co-occurrence (Weighted Projection)

What This Code Will Do This block creates a weighted projection of the women-only network. Edges are weighted by the number of shared events.

The goal is to show the total number of shared co-attendances with all other women.

Why This Matters The unweighted projection only tells us if two women ever attended the same event.

The weighted projection tells us: How often they attended events together.

This output shows the weighted degree for each woman in the projected women network.

Unlike the table which counted how many other women each person overlapped with, this code measures how often overlaps occurred.

How to Interpret the Output Higher values indicate stronger social integration.

For example:

- A value of 57 suggests frequent repeated interaction. For example, Theresa Anderson attended events with other women a combined total of 57 times.
- A value of 14 suggests limited overlap, such as Olivia Carleton and Flora Price.

This helps identify:

- Core influencers
- Strongly bonded subgroups
- Peripheral participants


```
[7]: # Module 4: The degree computed is weighted and counts the total number of
      ↪ shared contacts
W = bipartite.weighted_projected_graph(G, women)
print()
print("#Friend meetings, Member")
for w in women:
    print(f"{W.degree(w, weight='weight')} {w}")
```

```
#Friend meetings, Member
50 Evelyn Jefferson
45 Laura Mandeville
57 Theresa Anderson
46 Brenda Rogers
24 Charlotte McDowd
32 Frances Anderson
36 Eleanor Nye
31 Pearl Oglethorpe
40 Ruth DeSand
38 Verne Sanderson
33 Myra Liddel
37 Katherina Rogers
46 Sylvia Avondale
43 Nora Fayette
34 Helen Lloyd
24 Dorothy Murchison
14 Olivia Carleton
14 Flora Price
```

0.0.9 Module 5: Visualizing the Bipartite Structure

What This Code Will Do This block draws the bipartite network using a spring layout.

- Connected nodes move closer together
- Less connected nodes drift outward

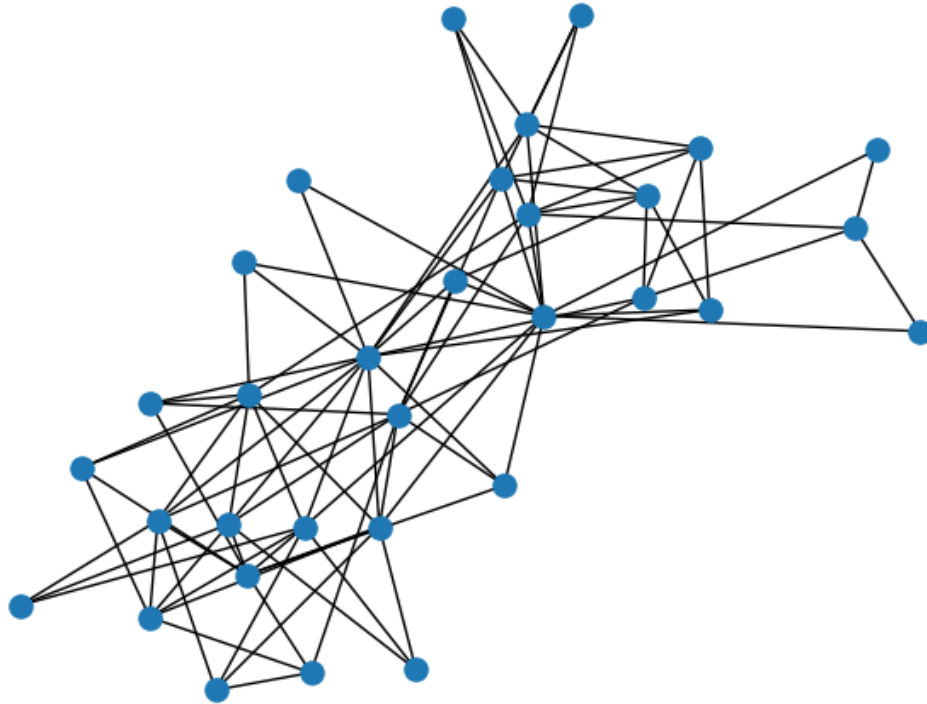
How to Interpret the Output In the middle of the graph there is a large cluster where many nodes are tightly packed together and connected by many overlapping edges. The graph shows many intersecting edges in the center, which is visual evidence of overlapping attendance. It indicates that many women attended the same events repeatedly, creating overlapping connections.

- Dense clusters in the center indicate highly connected actors
- Nodes on the edges indicate less socially integrated individuals
- Overall density indicates strong cohesion

The dataset shows a dense central core.

```
[8]: # Module 5: Visualizing the Bipartite Network
```

```
pos = nx.spring_layout(G, seed=648)
nx.draw(G, pos, node_size=90)
plt.show()
```



Module 5A: Enhanced Bipartite Visualization This module visualizes the original bipartite network while emphasizing the relative connectivity of the women.

In this graph, women are represented as orange nodes and events as blue nodes.

The shade of orange for each woman reflects how many other women she shares events with in the women-only projection. Darker orange nodes indicate women with more shared connections, meaning they attended events with many different participants and are more central within the social network. Lighter orange nodes represent women who attended events with fewer participants and appear more peripheral.

This visualization helps quickly identify the core participants of the social group, since darker orange nodes tend to cluster toward the center of the network where many connections overlap.

```
[9]: # Module 5A: Enhanced Bipartite Visualization
```

```
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
```

```

# Women projection to compute shared connections.
W = bipartite.projected_graph(G, women)
deg_values = {w: W.degree(w) for w in women}

# Discrete color mapping for women.
degree_color_map = {
    17: "#d95f0e", # darkest orange
    16: "#f16913",
    15: "#fd8d3c",
    12: "#fdae6b",
    11: "#fdd0a2"
}

# Assign node colors.
node_colors = []

for node in G.nodes():
    if node in women:
        node_colors.append(degree_color_map[deg_values[node]])
    else:
        node_colors.append("steelblue")

# Create node labels.
labels = {}

for w in women:
    initials = "".join([part[0] for part in w.split()])
    labels[w] = initials

for e in events:
    labels[e] = e

# Layout
pos = nx.spring_layout(G, seed=648)

# --- Draw network ---
plt.figure(figsize=(10,8))

nx.draw_networkx_nodes(
    G,
    pos,
    node_color=node_colors,
    node_size=900,
    edgecolors="black",
    linewidths=0.7

```

```

)

nx.draw_networkx_edges(
    G,
    pos,
    alpha=0.5
)

nx.draw_networkx_labels(
    G,
    pos,
    labels=labels,
    font_size=9
)

# --- Create legend ---
legend_elements = [

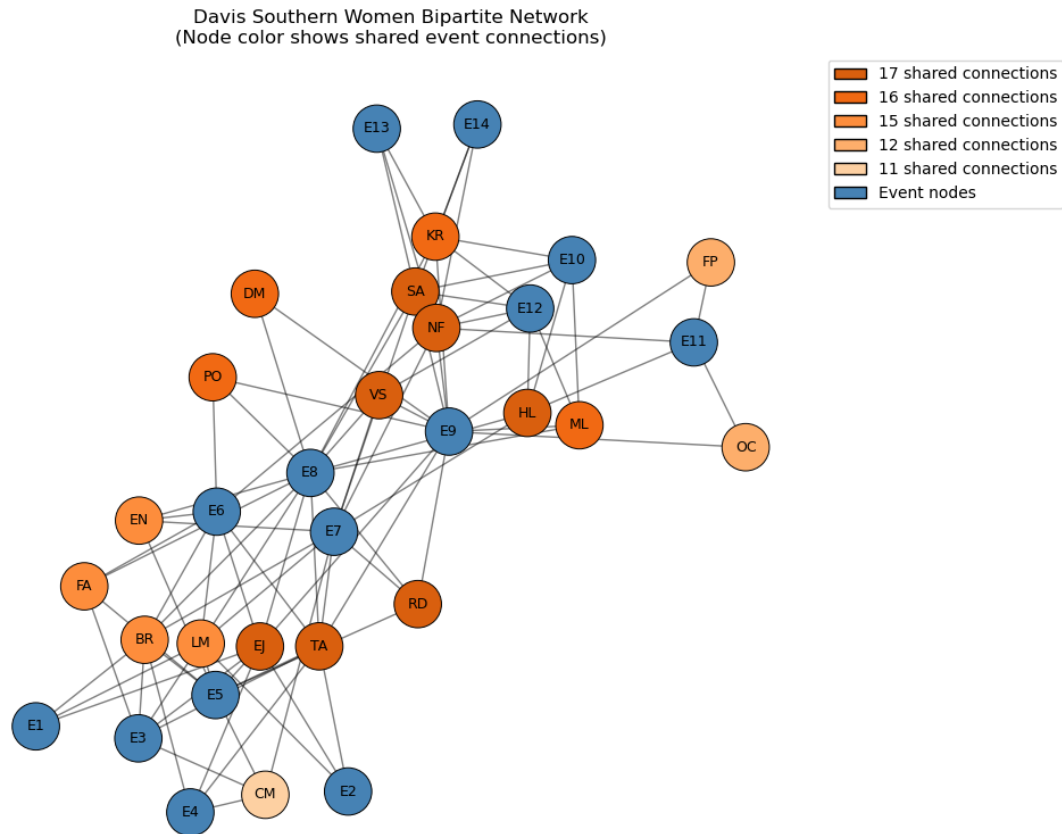
    Patch(facecolor="#d95f0e", edgecolor="black", label="17 shared_↵
connections"),
    Patch(facecolor="#f16913", edgecolor="black", label="16 shared_↵
connections"),
    Patch(facecolor="#fd8d3c", edgecolor="black", label="15 shared_↵
connections"),
    Patch(facecolor="#fdae6b", edgecolor="black", label="12 shared_↵
connections"),
    Patch(facecolor="#fdd0a2", edgecolor="black", label="11 shared_↵
connections"),
    Patch(facecolor="steelblue", edgecolor="black", label="Event nodes")

]

plt.legend(
    handles=legend_elements,
    loc="upper left",
    bbox_to_anchor=(1,1)
)

plt.title("Davis Southern Women Bipartite Network\n(Node color shows shared_↵
event connections)")
plt.axis("off")
plt.tight_layout()
plt.show()

```



0.0.10 Module 6: Event-Only Network Projection

In this code block, I decided to learn what events are connected.

What This Code Will Do This block projects the bipartite graph onto the events.

Two events are connected if they share at least one attendee.

The degree printed represents: The number of other events that share attendees.

Why This Matters This allows understanding which events:

- Are socially central
- Overlap heavily
- Cluster into themes or subgroups

How to Interpret the Output Higher degree events are popular, serve as social hubs, and connect multiple subgroups. The output 13 E6 below means Event E6, E7, E8 and E9 share attendees with 13 other events. Since there are 14 total events, this means E6 shares attendees with almost all but one event in the dataset.

```
[10]: # Module 6: Projecting onto Events
```

```
E = bipartite.projected_graph(G, events)

print("#Shared attendees, Event")
for e in events:
    print(E.degree(e), e)
```

```
#Shared attendees, Event
```

```
8 E1
8 E2
8 E3
8 E4
8 E5
13 E6
13 E7
13 E8
13 E9
8 E10
8 E11
8 E12
8 E13
8 E14
```

Module 6A: Woman and Event Network Visualization Enhancing Module 5A, Module 6A builds on it by shading in different tones of blue based on how many other events they share attendees with, as determined in the event-only projection from Module 6.

Darker blue nodes represent events that overlap with many other events and therefore function as central social hubs, while lighter blue nodes represent events with less overlap.

This visualization reveals how highly connected women interact through the most central events.

```
[11]: # Module 6A: Enhanced Bipartite Visualization with Dual Legends
```

```
import matplotlib.pyplot as plt
from matplotlib.patches import Patch

# Women projection (shared connections).
W = bipartite.projected_graph(G, women)
deg_values = {w: W.degree(w) for w in women}

# Event projection (shared attendees between events).
E = bipartite.projected_graph(G, events)
event_deg = {e: E.degree(e) for e in events}

# Discrete color mapping for women.
women_color_map = {
```

```

    17: "#d95f0e",
    16: "#f16913",
    15: "#fd8d3c",
    12: "#fdae6b",
    11: "#dd0a2"
}

# Discrete color mapping for events.
event_color_map = {
    13: "#08519c", # darker blue
    8: "#6baed6"   # lighter blue
}

# Assign node colors.
node_colors = []

for node in G.nodes():

    if node in women:
        node_colors.append(women_color_map[deg_values[node]])
    else:
        node_colors.append(event_color_map[event_deg[node]])

# Labels
labels = {}

for w in women:
    initials = "".join([part[0] for part in w.split()])
    labels[w] = initials

for e in events:
    labels[e] = e

# Layout
pos = nx.spring_layout(G, seed=648)

# --- Draw graph ---
plt.figure(figsize=(11,9))

nx.draw_networkx_nodes(
    G,
    pos,
    node_color=node_colors,
    node_size=950,
    edgecolors="black",
    linewidths=0.7
)

```

```

nx.draw_networkx_edges(
    G,
    pos,
    alpha=0.5
)

nx.draw_networkx_labels(
    G,
    pos,
    labels=labels,
    font_size=9
)

# Women legend
women_legend = [

    Patch(facecolor="#d95f0e", edgecolor="black", label="Women: 17 shared_↵
    ↵connections"),
    Patch(facecolor="#f16913", edgecolor="black", label="Women: 16 shared_↵
    ↵connections"),
    Patch(facecolor="#fd8d3c", edgecolor="black", label="Women: 15 shared_↵
    ↵connections"),
    Patch(facecolor="#fdae6b", edgecolor="black", label="Women: 12 shared_↵
    ↵connections"),
    Patch(facecolor="#fdd0a2", edgecolor="black", label="Women: 11 shared_↵
    ↵connections")
]

# Event legend
event_legend = [

    Patch(facecolor="#08519c", edgecolor="black", label="Events: 13 shared_↵
    ↵attendees"),
    Patch(facecolor="#6baed6", edgecolor="black", label="Events: 8 shared_↵
    ↵attendees")
]

# Add legends
legend1 = plt.legend(
    handles=women_legend,
    loc="upper left",
    bbox_to_anchor=(1.02,1),
    title="Women Connectivity"
)

```



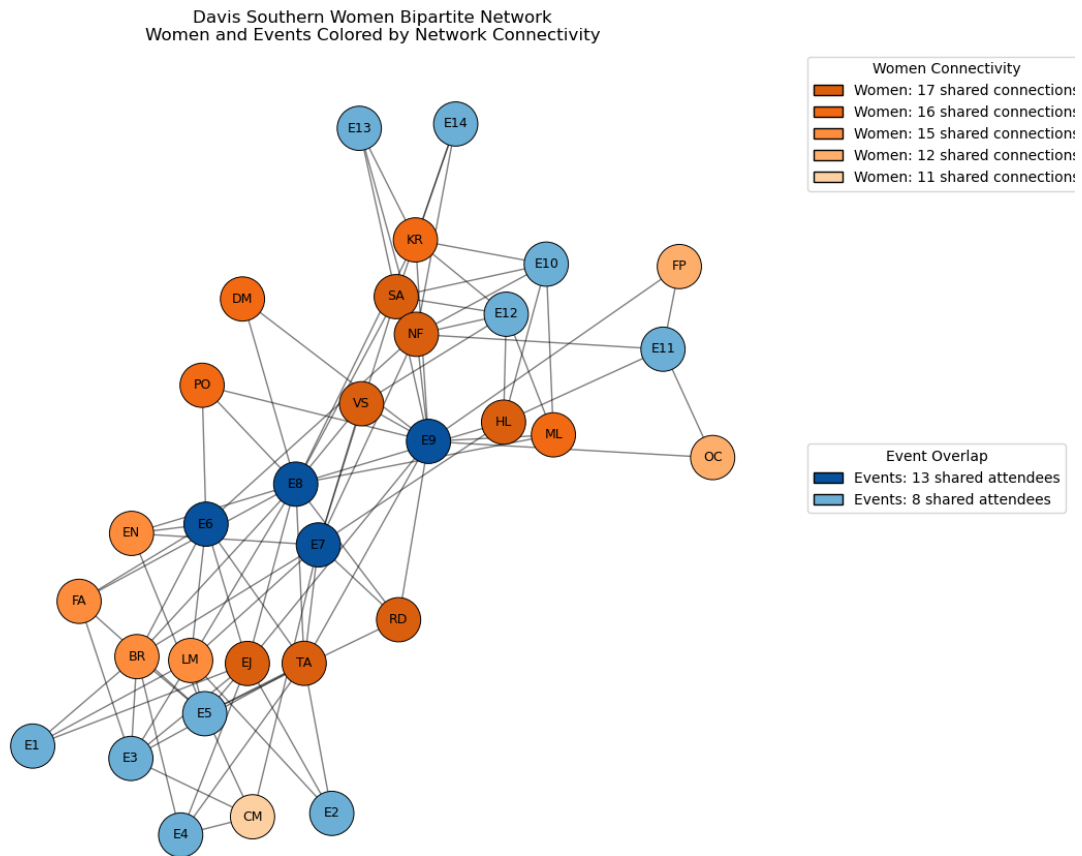
```

plt.gca().add_artist(legend1)

plt.legend(
    handles=event_legend,
    loc="upper left",
    bbox_to_anchor=(1.02,0.55),
    title="Event Overlap"
)

plt.title("Davis Southern Women Bipartite Network\nWomen and Events Colored by_\n↪Network Connectivity")
plt.axis("off")
plt.tight_layout()
plt.show()

```



Module 7: Betweenness Centrality (Identifying Social Bridges)

What This Code Will Do The code in this module calculates betweenness centrality for the women in the projected network.

Betweenness centrality measures how often a node lies on the shortest path between other nodes.

In simple terms, it identifies women who may act as bridges connecting different parts of the social network.

Why This Matters Betweenness centrality identifies which women help connect different groups.

A woman with high betweenness might not attend the most events, but she may link otherwise separate social circles. Higher betweenness values suggest that a woman positions herself on many shortest paths between other women, acting as a broker or bridge between social groups.

High betweenness individuals can play important roles in:

- connecting subgroups
- transmitting information
- linking events attended by different clusters of participants.

How to Interpret the Output Seven women have the highest betweenness values (0.010122): Evelyn Jefferson, Ruth DeSand, Helen Lloyd, Nora Fayette, Sylvia Avondale, Verne Sanderson, and Theresa Anderson. They likely attended events with multiple different subsets of the group, allowing them to connect otherwise separate social circles.

Three women have zero betweenness: Charlotte McDowd, Olivia Carleton, and Flora Price. They do not serve as bridges between groups. They appear to be more peripheral participants, attending events that overlap less with the broader social network.

The results suggest the network has a core group of socially active women who connect many others together. These individuals act as key connectors, helping link different participants through shared events. On the other hand, some women appear more peripheral, participating in fewer events or events attended by smaller subsets of the group.

```
[12]: # Module 7: Betweenness Centrality

import pandas as pd

# Use the women projection network
W = bipartite.projected_graph(G, women)

# Compute betweenness centrality
betweenness = nx.betweenness centrality(W)

# Convert to DataFrame
df_between = pd.DataFrame(
    [(w, betweenness[w]) for w in women],
    columns=["Woman", "Betweenness Centrality"]
)

# Sort descending
```

```
df_between = df_between.sort_values(
    by="Betweenness Centrality",
    ascending=False
).reset_index(drop=True)

df_between.index = df_between.index + 1
df_between.index.name = "Rank"

df_between
```

```
[12]:
```

	Woman	Betweenness Centrality
Rank		
1	Evelyn Jefferson	0.010122
2	Ruth DeSand	0.010122
3	Helen Lloyd	0.010122
4	Nora Fayette	0.010122
5	Sylvia Avondale	0.010122
6	Verne Sanderson	0.010122
7	Theresa Anderson	0.010122
8	Pearl Oglethorpe	0.005348
9	Myra Liddel	0.005348
10	Katherina Rogers	0.005348
11	Dorothy Murchison	0.005348
12	Eleanor Nye	0.002674
13	Frances Anderson	0.002674
14	Laura Mandeville	0.002674
15	Brenda Rogers	0.002674
16	Charlotte McDowd	0.000000
17	Olivia Carleton	0.000000
18	Flora Price	0.000000

Module 7A: Visualizing Betweenness with Node Size I expanded the graphs to include a betweenness centrality graph, showing how certain women act as bridges connecting different parts of the social network. The size of the women’s nodes reflects their betweenness centrality values.

Larger nodes indicate women who lie on more of the shortest paths connecting others, meaning they help link otherwise separate participants and maintain connectivity across the network. Smaller nodes represent women who are more embedded within a particular subgroup and do not frequently connect different parts of the network.

For example, Evelyn Jefferson (found in toward the lower middle of the graph) appears as one of the largest nodes because she has a high betweenness centrality value, indicating that she frequently connects different parts of the network. By contrast, someone like Flora Price (who has very low betweenness found in the upper right of the graph) appears much smaller because she participates in fewer overlapping event paths between other women. Flora Price may attend events and have direct connections, but she does not often serve as a link between different groups.

Evelyn Johnson, however, attends events that overlap with multiple clusters of participants. If Flora Price attends an event with Evelyn and another group of women also attends events with

Evelyn, then Evelyn becomes the connecting bridge between Flora and those other participants (as shown with E9). Evelyn's position helps link more peripheral individuals, like Flora, to the broader network, allowing connections to form between women who might otherwise remain socially separate.

```
[13]: # Module 7A: Visualizing Betweenness with Node Size

import math

# Compute betweenness centrality
bet = nx.betweenness centrality(W)

# Create stronger node size scaling
node_sizes = []

for node in G.nodes():

    if node in women:
        # nonlinear scaling to exaggerate differences
        size = 700 + (bet[node] ** 0.5) * 30000
        node_sizes.append(size)
    else:
        node_sizes.append(900)

# Slightly smaller figure
plt.figure(figsize=(15,12))

nx.draw_networkx_nodes(
    G,
    pos,
    node_color=node_colors,
    node_size=node_sizes,
    edgecolors="black",
    linewidths=0.8
)

nx.draw_networkx_edges(
    G,
    pos,
    alpha=0.5
)

nx.draw_networkx_labels(
    G,
    pos,
    labels=labels,
    font_size=9
```

```

)

plt.title("Betweenness Centrality Highlighted by Node Size", fontsize=16)
plt.axis("off")
plt.show()

```

Betweenness Centrality Highlighted by Node Size

